

Les Sémaphores

Introduction

Les solutions au problème de l'exclusion mutuelle présentées jusqu'à présent dans ce cours sont difficiles à réaliser et à comprendre puisque on ne peut pas trancher si une variable est utilisée pour assurer l'exclusion mutuelle ou pour faire autre chose.

De plus ces solutions sont basées sur le principe d'attente active : un processus qui ne peut pas progresser dans son protocole d'entrée, occupe inutilement le processeur. Un processeur peut être utilisé d'une façon plus productive s'il n'attend pas activement qu'une condition change d'état dans un processus.

Définition des sémaphores

Le principe fondamental des sémaphores est basé sur le fait que deux ou plusieurs peuvent coopérer en utilisant des **signaux** simples. On peut considérer un sémaphore comme étant une variable qui contient une valeur entière et sur laquelle trois (3) opérations sont définies :

- Un sémaphore doit être initialisé à une valeur entière non négative
- Pour recevoir un signal via un sémaphore **S**, un processus exécute la fonction primitive **wait(S)**. Cette fonction décrémente la valeur du sémaphore ; si la valeur devient négative, alors le processus qui a exécuté **wait** doit être bloqué.
- Pour transmettre un signal via un sémaphore **S**, un processus exécute la fonction primitive **signal(S)**. Cette fonction incrémente la valeur du sémaphore ; si la valeur (après incrémentation) n'est pas positive, alors un processus bloqué par **wait** et débloqué.

Remarque :

La désignation des opérations peuvent changer d'un système à un autre :

- **P** (Proberen : tester) ou **Down** pour l'opération **wait**
- **V** (Verhogen : incrémenter) ou **Up** pour l'opération **signal**

Déclaration des sémaphores

Le type sémaphore est défini comme suit :

```
struct semaphore{  
    int count ;  
    QueueType queue ;  
}
```

Les opérations sur une variable de type sémaphore :

```
void wait(semaphore s){
    s.count -- ;
    if( s.count<0 ) {
        Placer ce processus dans s.queue;
        Bloquer ce processus ;
    }

}

void signal(semaphore s){
    s.count++ ;
    if( s.count<= 0 ) {
        Retirer un processus p de s.queue;
        Placer le processus p dans la liste des processus « prêts » ;
    }

}
```

Propriétés des sémaphores :

- Les opérations wait et signal sont atomiques (une opération wait ou signal est exécutée sans possibilité d'interruption)
- Les opérations wait et signal sont en Exclusion Mutuelle, elles ne sont jamais exécutées en même temps; de même que deux opérations wait ou deux opérations signal sur le même variable sémaphore.
- Quand $s.count \geq 0$, le nombre de processus qui peuvent exécuter wait(s) sans être bloqué est $s.count$
- Quand $s.count < 0$, le nombre de processus en attente dans la file $s.queue$ est $|s.count|$
- La déclaration d'une variable de type sémaphore doit toujours inclure une initialisation de cette variable à une valeur entière non négative.
- Un sémaphore d'exclusion mutuelle (mutex) est initialisé à 1
- Un sémaphore dont la définition inclut l'application d'une politique FIFO au niveau de la gestion de $s.queue$ est appelé un sémaphore **fort**.
- Un sémaphore dont la définition ne précise pas la politique de gestion de $s.queue$ est appelé un sémaphore **faible**
- Sauf exception, nous supposons que les sémaphores que nous utilisons sont des sémaphores forts.

Utilisation de sémaphore pour la réalisation d'exclusion mutuelle

Soient N processus en exclusion mutuelle sur un objet commun. On utilise un sémaphore *mutex* initialisé à 1.

```
semaphore mutex = 1 ;
```

```
void Pi ( ){  
    while(1){  
        .....  
        wait(mutex) ;  
        SC  
        signal(mutex) ;  
        .....  
    }  
}
```

Problème du Producteur Consommateur

Énoncé du problème

Un processus producteur place (dépose) des éléments dans un tampon (buffer) pour qu'un processus consommateur puisse les retirer et les traiter.

Ce schéma constitue une communication entre 2 processus, le producteur étant l'émetteur du message et le consommateur, le destinataire du message.

Les producteurs et les consommateurs doivent se coordonner afin de respecter les contraintes suivantes :

- Les éléments contenus dans le tampon ne sont consommés qu'une seule fois ;
- Les éléments du tampon sont consommés selon leur ordre de production ;
- Si le tampon est plein, un producteur doit attendre la libération d'un élément du tampon.
- Si le tampon est vide, un consommateur doit attendre le dépôt d'un nouvel élément.
- Empêcher le chevauchement des opérations sur le tampon.

Problème de Producteur-Consommateur avec un tampon infini

De manière abstraite, on peut définir les codes des deux processus comme suit :

Producteur : while(true){ /* produire l'élément v */ B[in] = v ; in++ ; }	Consommateur : while(true){ while(in ≤ out); w = B[out]; out++; /* consommer l'élément w */ }
---	--

Première solution

int n ; /*nombre des éléments dans le tampon */

semaphore mutex = 1 ; /* EM sur l'accès au tampon et à la variable n */

semaphore delay = 0 ; /*force le consommateur à attendre quand le tampon est vide */

<pre>void Producteur () { while(1) { produire () ; wait (mutex) ; déposer () ; n++ ; if(n == 1) signal(delay) ; signal(mutex); } }</pre>	<pre>void Consommateur () { wait(delay) ; while(1){ wait (mutex) ; retirer () ; n-- ; signal(mutex); consommer () ; if(n == 0) wait (delay) ; } }</pre>	<pre>int main () { n = 0; parbegin(producteur(),..., producteur(),consommateur()); return 0; }</pre>
--	---	---

Le scenario suivant montre que la solution est incorrecte, car le nombre d'éléments dans un tampon ou un tableau ne doit jamais avoir une valeur négative.

Producteur	Consommateur	mutex	N	delay
		1	0	0
wait (mutex)		0	0	0
n++		0	1	0
if(n == 1) signal(delay)		0	1	1
signal(mutex)		1	1	1
	wait(delay)	1	1	0
	wait (mutex)	0	1	0
	n--	0	0	0
	signal(mutex)	1	0	0
wait (mutex)		0	0	0
n++		0	1	0
if(n == 1) signal(delay)		0	1	1
signal(mutex)		1	1	1
	if(n == 0) wait (delay)	1	1	1
	wait (mutex)	0	1	1
	n--	0	0	1
	signal(mutex)	1	0	1
	if(n == 0) wait (delay)	1	0	0
	wait (mutex)	0	0	0
	n--	0	-1	0
	signal(mutex)			

Deuxième solution

```
int n ; /*nombre des éléments dans le tampon */
semaphore mutex = 1 ; /* EM sur l'accès au tampon et à la variable n */
semaphore delay = 0 ; /*force le consommateur à attendre quand le tampon est vide */
```

<pre>void Producteur () { while(1) { produire () ; wait (mutex) ; déposer () ; n++ ; if(n == 1) signal(delay) ; signal(mutex); } }</pre>	<pre>void Consommateur () { int m ; /* variable locale*/ wait(delay) ; while(1){ wait (mutex) ; retirer () ; n-- ; m = n; signal(mutex); consommer () ; if(m == 0) wait (delay) ; } }</pre>	<pre>int main () { n = 0; parbegin(producteur(),..., producteur(),consommateur()); return 0; }</pre>
--	---	---

Problème de Producteur-Consommateur avec un tampon fini (limité)

Le tampon peut contenir un nombre maximum d'éléments et ces éléments sont organisés en liste circulaire et les valeurs des indices doivent être exprimées par le modulo de la taille du tampon. Un élément du tampon, dont le contenu a été consommé, peut à nouveau servir.

De manière abstraite, on peut définir les codes des deux processus comme suit :

<pre>Producteur : while(true){ /* produire l'élément v */ while(count == n) ; B[in] = v ; in = (in+1) % n ; count ++; }</pre>	<pre>Consommateur : while(true){ while(count == 0); w = B[out]; out = (out+1) % n ; count --; /* consommer l'élément w */ }</pre>
---	---

Solution pour un tampon de taille limité

```
const int size = 100 ; /*taille du tampon */
semaphore mutex = 1 ; /* EM sur l'accès au tampon */
semaphore n = 0 ; /*pas de retrait d'un tampon vide */
semaphore e = size ; /*pas de dépôt dans un tampon plein */
```

<pre>void Producteur () { while(1) { produire () ; wait (e) ; wait (mutex) ; deposer () ; signal (mutex); signal (n); } }</pre>	<pre>void Consommateur () { while(1) { wait (n) ; wait (mutex) ; retirer (); signal (mutex); signal (e); consommer () ; } }</pre>	<pre>int main () { parbegin(producteur(),..., producteur(),consommateur()); return 0; }</pre>
---	---	--

Problème de Lecteurs-Rédacteurs

Les situations de type lecteurs-rédacteurs correspondent à l'accès concurrent à une ressource qui est accédée en lecture par certains processus, et en écriture par d'autres. Un exemple typique est l'accès à un fichier, qui peut être ouvert simultanément en lecture par plusieurs processus, mais en écriture par un et un seul.

Énoncé du problème

Un ensemble de processus (lecteurs et rédacteurs) se partagent des données. Les lecteurs ne font que lire les données, tandis que les rédacteurs peuvent également les modifier (écrire). Les opérations de lecture peuvent être concurrentes. Plusieurs processus peuvent accéder à un même fichier en lecture sans risque de trouver le fichier dans un état incohérent.

Les écritures sont, par contre, critiques. Si deux processus modifient un fichier simultanément, les données de ce fichier risquent d'être corrompues. Et de même, si un processus modifie un fichier alors qu'un autre l'accède en lecture, ce dernier peut se retrouver avec des données incohérentes.

Alors, les accès en écriture doivent être effectués en exclusion mutuelle : une écriture ne peut être réalisée si une autre écriture est en cours, une lecture ne peut être faite tant qu'une écriture est en cours, et une écriture ne peut débuter que si aucune lecture n'est en cours.

Pour résumer, à un moment donné on ne doit avoir que l'une des deux situations suivantes :

- Un seul rédacteur est en train de modifier le fichier
- Un ou plusieurs lecteurs sont en train de lire le contenu du fichier

Le problème des lecteurs-rédacteurs est plus général qu'un simple problème d'exclusion mutuelle, de par ces contraintes. La solution à ce problème n'est pas unique. Plusieurs solutions peuvent en effet être proposées, en fonction des priorités choisies :

1. Priorité aux lecteurs (famine possible des rédacteurs)
2. Priorité aux lecteurs si un lecteur a déjà accès à la ressource (famine possible des lecteurs)
3. Priorité aux rédacteurs (famine possible des lecteurs)
4. Accès aux données selon les ordres des arrivées. Toutes les demandes des lecteurs qui se suivent sont satisfaites en même temps.

Nous nous intéresserons dans ce cours au cas où on donne une certaine priorité aux Lecteurs. Autrement dit, aucun Lecteur n'attend, à moins qu'un rédacteur n'ait déjà obtenu l'autorisation pour utiliser le fichier. Les autres cas seront traités au TD.

semaphore mutex = 1 ; /* EM sur l'accès à la variable ReadCount*/ semaphore wrt = 1 ; /* EM sur l'accès en écriture de fichier */ int ReadCount = 0 ; /* le nombre de Lecteurs actuellement dans le fichier */		
void Lecteur (){ wait(mutex) ; ReadCount ++; if (ReadCount ==1) wait(wrt) signal(mutex) ; /* Lire le fichier */ wait(mutex) ; ReadCount --; if (ReadCount ==0) signal(wrt) signal(mutex); }	void Redacteur (){ wait(wrt) /* Écrire dans le fichier */ signal (wrt) }	int main () { parbegin(Lecteur(), Redacteur()); return 0; }

Les sémaphores Binaires

- Les sémaphores binaires sont des sémaphores qui ne peuvent prendre que les valeurs **0** et **1**.
- Ils sont utilisés pour mettre en œuvre les verrous en utilisant un mécanisme de signalisation pour obtenir une exclusion mutuelle.
- Si la valeur du sémaphore est **0**, cela signifie qu'il est verrouillé, donc le **verrou** n'est pas disponible.
- Si la valeur du sémaphore est **1**, cela signifie qu'il est déverrouillé, donc le **verrou** est disponible.

Structure de semaphore binaire

```
struct binary_semaphore{
    enum {zero, one} value ;
    queueType queue;
};
```

Un semaphore binaire peut être initialisé à 0 ou 1.

Les opérations primitives

L'opération **waitB** contrôle la valeur du semaphore.

- Si la valeur est nulle, alors le processus qui exécute waitB est bloqué.
- Si la valeur est 1, alors la valeur est changée est mise à zéro et le processus continue l'exécution.

```

void waitB(binary_semaphore s){
    if (s.value == one)
        s.value = zero;
    else {
        /* Placer ce processus dans s.queue */
        /* Bloquer ce processus */
    }
}

```

L'opération **signalB** vérifie si des processus sont bloqués sur ce semaphore (la valeur du semaphore est égale à zéro).

- Si ainsi, alors un processus bloqué sur l'opération du waitB est débloqué.
- Si aucun processus n'est bloqué, alors la valeur du semaphore est changée est mise à un (1).

```

void signalB(binary_semaphore s){
    if (s.queue is empty())
        s.value = one;
    else {
        /* Retirer un processus P de s.queue */
        /* Placer le processus P dans la liste des processus prêt */
    }
}

```

L'exclusion mutuelle pour l'utilisation d'une ressource critique peut s'exprimer comme suit :

```

semaphore mutex = 1
void Pi ( ) { /* i=1,n */
    waitB(mutex)
    /* section critique */
    signalB(mutex)
}

```

Remarque :

- Les primitives **waitB** et **signalB** sont aussi dénommées **Lock** et **Unlock** respectivement.
- L'utilisation de **waitB** et **signalB** à la place de **wait** et **signal** ne donne pas toujours le même résultat